Objectives

- To perform message passing between related processes (parent and child) using an unnamed pipe.
- To perform message passing among a chain of processes using unnamed pipes.
- To perform message passing among fans of processes using unnamed pipes.
- To perform message passing between unrelated processes using a named pipe (Special File).

Pre-Lab Theory

1. ***pipe() synopsis:***

int pipe(int fd[])

Description:

The inter-process communication channel is created and two descriptors are allocated and stored in the 2-element integer array provided as an argument in the pipe() function.

file descriptor fd[0]:

Represents a handle for reading from the pipe. It is the index of the entry created in the process file table.

file descriptor fd[1]:

Represents a handle for writing to the pipe. It is the index of the entry created in the process file table.

Default Descriptors in the process file table with symbolic names:

Standard output: *STDOUT_FILENO*

Standard input: *STDIN_FILENO*

Standard Error: *STDERR_FILENO*

return*: Negative value if the pipe(special file) is not created in case of an error*

Named Pipe: Special File

#include <sys/stat.h>

*int mkfifo(const char *path, mode_t mode);*

Description:

The mkfifo() function shall create a new FIFO special file named by the pathname pointed to by the path. The file permission bits of the new FIFO shall be initialized from mode. The file permission bits of the mode argument shall be modified by the process file creation mask.

***Return**: 0 on successful creation of named pipe and -1 in case of an*

error Example Usage:

```
#include <sys/types.h>
#include <sys/stat.h>


int status;
...
status = mkfifo("/home/OS/Lab4/myfifo", S_IWUSR | S_IRUSR |
    S_IRGRP | S_IROTH);

fd = Open(("/home/OS/Lab4/myfifo",O_RDWR);
read(fd,….,….)
write(fd,…,….)
```

In-Lab Tasks

header file*: unistd.h, stdio.h*

System call: pipe(), fork() read(), write(), open(), close()

Task1(a):

Include the header files and the following line of code in your program. Compile and run the code and describe its behavior.

```
Int fd[2];
char buffer_p[] = “Welcome”; char
buffer_c[10];
If (Pipe(fd) > 0)
     Printf(“Pipe is successfully created\n”);
Else
     Printf(“Error”); exit();
If ((pid = fork()) > 0)
     Printf(“Child Process created\n”) write(fd[1],
     buffer, sizeof(buffer);
Else if(pid == 0)
     read(fd[0], buffer_c, sizeof(buffer);
     printf(“%d Received %s from Parent Process %d,getpid(),
buffer_c,getppid() );
```

Brief the working/output:

```
muhammad-ahmad@latitude-7490:~/lab4$ ./task1
Pipe is successfully created
Child process created
5337 received 'Welcome' from Parent Process 5336
```

→ the pipe is created successfully[as pipe(fd)return greater then zero]
→ the parent and child build a one way communication channel from parernt
→ child

Task 1(b):

Include the header files, declare the required variables, extend the code in Task 1(a) to create a two way communication between parent and child process. The child process after receiving and displayed the "Welcome" message from the parent send the "Thanks" message to the parent. Parent displays the message received from the child process to its Standard Output.

Parent Output format:

<process_id> received Thanks from <child_process_id>

Child Output format:

<process_id> received Welcome from <parent_process_id>

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

int main() {
    int fd1[2];       // Pipe1: Parent -> Child
    int fd2[2];       // Pipe2: Child -> Parent
    pid_t pid;
    char buffer_p[] = "Welcome";
    char buffer_c[20];
    char buff_c[] = "Thanks";

    // Create pipe1
    if (pipe(fd1) < 0) {
        perror("Pipe1 creation failed");
        exit(1);
    }

    // Create pipe2
    if (pipe(fd2) < 0) {
        perror("Pipe2 creation failed");
        exit(1);
    }

    pid = fork();

    if (pid > 0) {
        // Parent process

        // Close unused pipe ends
        close(fd1[0]); // Close read end of pipe1
```

```
        close(fd2[1]); // Close write end of pipe2

        // Send message to child
        write(fd1[1], buffer_p, strlen(buffer_p) + 1);

        // Receive message from child
        read(fd2[0], buffer_p, sizeof(buffer_p));
        printf("%d received '%s' from Child Process %d\n", getpid(),
buffer_p, pid);

        // Close used pipe ends
        close(fd1[1]);
        close(fd2[0]);
    }
    else if (pid == 0) {
        // Child process

        // Close unused pipe ends
        close(fd1[1]); // Close write end of pipe1
        close(fd2[0]); // Close read end of pipe2

        // Receive message from parent
        read(fd1[0], buffer_c, sizeof(buffer_c));
        printf("%d received '%s' from Parent Process %d\n", getpid(),
buffer_c, getppid());

        // Send message back to parent
        write(fd2[1], buff_c, strlen(buff_c) + 1);

        // Close used pipe ends
        close(fd1[0]);
        close(fd2[1]);
    }
    else {
        perror("Fork failed");
        exit(1);
    }
    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab4$ ./task2
216386 received 'Welcome' from Parent Process 216385
216385 received 'Thanks' from Child Process 216386
```

Task 2(a):

Include the header files, declare the required variables, and extend the code in Task 1(a), and Task 1(b) to create a two-way message relay system between a chain of processes. Every child process after receiving and displaying the "Welcome" message from the parent replies with the "Thanks" message and forwards the "Welcome" message to the ***child process in the chain****.*

Parent Output format:
<process_id> received Thanks from <child_process_id>
Child Output format:

<process_id> received Welcome from <parent_process_id>

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

int main() {
    int fd1[2];       // Pipe1: Parent -> Child
    int fd2[2];       // Pipe2: Child -> Parent
    pid_t pid;
    char buffer_p[] = "Welcome";
    char buffer_c[20];
    char buff_c[] = "Thanks";

    // Create pipe1
    if (pipe(fd1) < 0) {
        perror("Pipe1 creation failed");
        exit(1);
    }
    // Create pipe2
    if (pipe(fd2) < 0) {
        perror("Pipe2 creation failed");
        exit(1);
    }
    for (int i=0 ; i <4 ; i++){
        pid = fork();
        if (pid > 0) {
            // Parent process
            // Close unused pipe ends
            close(fd1[0]); // Close read end of pipe1
            close(fd2[1]); // Close write end of pipe2
```

```
            // Send message to child
            write(fd1[1], buffer_p, strlen(buffer_p) + 1);

            // Receive message from child
            read(fd2[0], buffer_p, sizeof(buffer_p));
            printf("%d received '%s' from Child Process %d\n", getpid(),
buffer_p, pid);
            // Close used pipe ends
            close(fd1[1]);
            close(fd2[0]);
            break;
        }
        else if (pid == 0) {
            // Child process
            // Close unused pipe ends
            close(fd1[1]); // Close write end of pipe1
            close(fd2[0]); // Close read end of pipe2
            // Receive message from parent
            read(fd1[0], buffer_c, sizeof(buffer_c));
            printf("%d received '%s' from Parent Process %d\n", getpid(),
buffer_c, getppid());
            // Send message back to parent
            write(fd2[1], buff_c, strlen(buff_c) + 1);
            // Close used pipe ends
            close(fd1[0]);
            close(fd2[1]);
            continue;
        }
        else {
            perror("Fork failed");
            exit(1);
        }
    }
    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab4$ ./task3
217133 received 'Welcome' from Parent Process 217132
217133 received 'Welcome' from Child Process 217134
217132 received 'Thanks' from Child Process 217133
217134 received 'Welcome' from Parent Process 217133
217134 received 'Welcome' from Child Process 217135
217135 received 'Welcome' from Parent Process 217134
217135 received 'Welcome' from Child Process 217136
217136 received 'Welcome' from Parent Process 217135
```

Task 2(b):

Include the header files, declare the required variables, and extend the code in Task 1(a), and Task 1(b) to create a two-way message relay system between a ***Fan of processes****. Every child process after receiving and displaying the "Welcome" message from the parent replies with the "Thanks" message. The parent process displays all the messages received with the sender process ID.*

Parent Output Format:
<process_id> received Thanks from <Child_process_id>
Child Output Format:
<process_id> received Welcome from <Parent_Process_Id>

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

int main() {
    int fd1[2];       // Pipe1: Parent -> Child
    int fd2[2];       // Pipe2: Child -> Parent
    pid_t pid;
    char buffer_p[] = "Welcome";
    char buffer_c[20];
    char buff_c[] = "Thanks";

    // Create pipe1
    if (pipe(fd1) < 0) {
        perror("Pipe1 creation failed");
        exit(1);
    }
    // Create pipe2
    if (pipe(fd2) < 0) {
        perror("Pipe2 creation failed");
        exit(1);
    }
    for (int i=0 ; i <4 ; i++){
```

```
        pid = fork();
        if (pid > 0) {
            // Parent process
            // Close unused pipe ends
            close(fd1[0]); // Close read end of pipe1
            close(fd2[1]); // Close write end of pipe2

            // Send message to child
            write(fd1[1], buffer_p, strlen(buffer_p) + 1);

            // Receive message from child
            read(fd2[0], buffer_p, sizeof(buffer_p));
            printf("%d received '%s' from Child Process %d\n", getpid(),
buffer_p, pid);
            // Close used pipe ends
            close(fd1[1]);
            close(fd2[0]);
            continue;
        }
        else if (pid == 0) {
            // Child process
            // Close unused pipe ends
            close(fd1[1]); // Close write end of pipe1
            close(fd2[0]); // Close read end of pipe2
            // Receive message from parent
            read(fd1[0], buffer_c, sizeof(buffer_c));
            printf("%d received '%s' from Parent Process %d\n", getpid(),
buffer_c, getppid());
            // Send message back to parent
            write(fd2[1], buff_c, strlen(buff_c) + 1);
            // Close used pipe ends
            close(fd1[0]);
            close(fd2[1]);
            break;
        }
        else {
            perror("Fork failed");
```

```
            exit(1);
        }
    }
    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab4$ ./task4
5425 received 'Welcome' from Parent Process 5424
5424 received 'Thanks' from Child Process 5425
5424 received 'Thanks' from Child Process 5426
5426 received '' from Parent Process 5424
5424 received 'Thanks' from Child Process 5427
5427 received '' from Parent Process 5424
5424 received 'Thanks' from Child Process 5428
5428 received '' from Parent Process 5424
```

Task 3:

Create a two-way communication channel between two unrelated processes (Client and Server) using a named pipe (Fifo). The Client Process gets some text from standard input and sends it to the server process with its process Id. The server process displays the message received from the client and replies with a "Welcome" Message with its process id. The client displays the "Welcome" message received from the server and server process id.

Both Processes use the same named pipe for message exchange.

Client Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define FIFO_CS "client_to_server"
#define FIFO_SC "server_to_client"

int main() {
    int fd_cs, fd_sc;
    char buffer[512];

    // Open FIFOs
    fd_cs = open(FIFO_CS, O_WRONLY);
```

```
    fd_sc = open(FIFO_SC, O_RDONLY);

    if (fd_cs < 0 || fd_sc < 0) {
        perror("open");
        exit(1);
    }

    printf("Enter a message: ");
    fgets(buffer, sizeof(buffer), stdin);
    buffer[strcspn(buffer, "\n")] = '\0'; // remove newline

    // Send to server
    char message[600];
    snprintf(message, sizeof(message), "Client[PID=%d]: %s", getpid(),
buffer);
    write(fd_cs, message, strlen(message));

    // Wait for server reply
    int n = read(fd_sc, buffer, sizeof(buffer)-1);
    if (n > 0) {
        buffer[n] = '\0';
        printf("Client received: %s\n", buffer);
    }

    close(fd_cs);
    close(fd_sc);
    return 0;
}
```

Server Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define FIFO_CS "client_to_server"
#define FIFO_SC "server_to_client"

int main() {
    int fd_cs, fd_sc;
    char buffer[512];

    // Create FIFOs if not exist
    mkfifo(FIFO_CS, 0666);
```

```
    mkfifo(FIFO_SC, 0666);

    printf("Server started... Waiting for messages.\n");

    // Open FIFOs
    fd_cs = open(FIFO_CS, O_RDONLY);
    fd_sc = open(FIFO_SC, O_WRONLY);

    if (fd_cs < 0 || fd_sc < 0) {
        perror("open");
        exit(1);
    }

    while (1) {
        // Read from client
        int n = read(fd_cs, buffer, sizeof(buffer)-1);
        if (n > 0) {
            buffer[n] = '\0';
            printf("Server received: %s\n", buffer);

            // Reply to client
            char reply[256];
            snprintf(reply, sizeof(reply), "Welcome from Server [PID=
%d]", getpid());
            write(fd_sc, reply, strlen(reply));
        }
    }

    close(fd_cs);
    close(fd_sc);
    unlink(FIFO_CS);
    unlink(FIFO_SC);
    return 0;
}
```

Server Side Output:

```
muhammad-ahmad@latitude-7490:~/lab4/cs$ ./server
Server started... Waiting for messages.
Server received: Client[PID=242744]: sfsdf
```

Client Side output:

```
muhammad-ahmad@latitude-7490:~/lab4/cs$ ./client
Enter a message: sfsdf
Client received: Welcome from Server [PID=242711]
```

Task 3b

Do the same task as above but make a one way connection using single fifo

Servers Code:

```
#include <stdio.h>      // For input/output functions (printf, perror)
#include <stdlib.h>     // For exit()
#include <string.h>     // For string operations (strlen, snprintf)
#include <unistd.h>     // For POSIX API (read, write, close, getpid,
unlink)
#include <fcntl.h>      // For file control (open, O_RDWR)
#include <sys/stat.h>   // For mkfifo and file permissions
#include <errno.h>      // For errno to check specific errors

#define FIFO_NAME "myfifo"  // Name of the FIFO file (named pipe)
#define BUFFER_SIZE 256     // Buffer size for reading messages

int main() {
    int fd;                     // File descriptor for the FIFO
    char buffer[BUFFER_SIZE];  // Buffer for incoming messages

    // Create FIFO if it doesn't exist
    // Permissions 0666 allow read/write for all users
    if (mkfifo(FIFO_NAME, 0666) == -1 && errno != EEXIST) {
        perror("Failed to create FIFO");
        exit(EXIT_FAILURE);  // Exit on failure (except if FIFO already
exists)
    }

    printf("Server started... Waiting for messages.\n");

    // Open FIFO for read/write to prevent blocking issues
    fd = open(FIFO_NAME, O_RDWR);
    if (fd < 0) {
        perror("Failed to open FIFO");
        unlink(FIFO_NAME);  // Clean up if open fails
        exit(EXIT_FAILURE);
    }

    // Main loop to handle client messages
    while (1) {
        // Read message from client (blocks until data is available)
        ssize_t bytes_read = read(fd, buffer, BUFFER_SIZE - 1);
        if (bytes_read > 0) {
            buffer[bytes_read] = '\0';  // Null-terminate for safe
printing
            printf("Server received: %s\n", buffer);
```

```
            // Prepare and send response with server PID
            char reply[BUFFER_SIZE];
            snprintf(reply, sizeof(reply), "Welcome from Server [PID=
%d]", getpid());
            if (write(fd, reply, strlen(reply)) < 0) {
                perror("Failed to write response");
            }
        } else if (bytes_read < 0) {
            perror("Failed to read from FIFO");
            break;  // Exit loop on read error
        }
        // bytes_read == 0 (EOF) is ignored as FIFO stays open with
O_RDWR
    }

    // Cleanup (unreachable unless loop is broken, e.g., by signal)
    close(fd);
    unlink(FIFO_NAME);  // Remove FIFO from filesystem
    return 0;
}
```

Client Code:

```
#include <stdio.h>
#include <stdlib.h>     // For exit()
#include <string.h>
#include <unistd.h>     // For POSIX API (read, write, close, getpid)
#include <fcntl.h>      // For file control (open, O_RDWR)
#include <sys/stat.h>   // For file status
#include <errno.h>      // For errno handling

#define FIFO_NAME "myfifo"  // Name of the FIFO file (must match server)
#define BUFFER_SIZE 512     // Buffer size for input and messages

int main() {
    int fd;                     // File descriptor for the FIFO
    char input_buffer[BUFFER_SIZE];  // Buffer for user input and server
reply
    char message[BUFFER_SIZE + 64];  // Buffer for formatted message
(input + PID)


    // Open FIFO for read/write
    fd = open(FIFO_NAME, O_RDWR);
    if (fd < 0) {
        perror("Failed to open FIFO");
```

```
        exit(EXIT_FAILURE);
    }

    // Get user input
    printf("Enter a message: ");
    if (fgets(input_buffer, BUFFER_SIZE, stdin) == NULL) {
        perror("Failed to read input");
        close(fd);
        exit(EXIT_FAILURE);
    }
    input_buffer[strcspn(input_buffer, "\n")] = '\0';  // Remove newline

    // Format message with client PID
    snprintf(message, sizeof(message), "Client[PID=%d]: %s", getpid(),
input_buffer);

    // Write message to FIFO
    if (write(fd, message, strlen(message)) < 0) {
        perror("Failed to write to FIFO");
        close(fd);
        exit(EXIT_FAILURE);
    }

    // Read server response
    ssize_t bytes_read = read(fd, input_buffer, BUFFER_SIZE - 1);
    if (bytes_read > 0) {
        input_buffer[bytes_read] = '\0';  // Null-terminate for printing
        printf("Client received: %s\n", input_buffer);
    } else if (bytes_read < 0) {
        perror("Failed to read from FIFO");
    }

    // Cleanup
    close(fd);
    return 0;
}
```

Task 4 *: make a redirection [ls -l | sort -n +4]*

→ we use ipc of child and parent process (parent process runs ls -l & child process run sort)

→ parent redirects it output to child instead of standard output & child took his input from parent instead of standard input

hint:

→ parent processes file:

. stdin

. stdout [overwrite with pipe write]

. stderr

.pipe read

.pipewrite

→ child process file:

. stdin [overwrite with piperead]

. stdout

. stderr

.pipe read

.pipewrite

→ use dup2 to change file discriptor

→ also close unused fds

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pid_t pid;
    // Create pipe
    if (pipe(fd) < 0) {
        perror("pipe");
        exit(1);
    }
```

```
    // Fork
    pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }
    if (pid == 0) {
        // Child process: executes "sort -n +4"
        close(fd[1]);  // close unused write end
        // Redirect stdin to read end of pipe
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        execlp("sort", "sort", "-n", "+4", NULL); // the first sort is the
        perror("execlp sort"); // only reached if execlp fails
        exit(1);
    } else {
        // Parent process: executes "ls -l"
        close(fd[0]);  // close unused read end
        // Redirect stdout to write end of pipe
        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);
        execlp("ls", "ls", "-l", NULL);
        perror("execlp ls"); // only reached if execlp fails
        exit(1);
    }
    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab4/pipe$ ./ls-l
total 56
-rw-rw-r-- 1 muhammad-ahmad muhammad-ahmad  1048 Oct  2 15:42 ls-l.c
drwxrwxr-x 2 muhammad-ahmad muhammad-ahmad  4096 Sep 30 16:41 Lab4 (2)
-rwxrwxr-x 1 muhammad-ahmad muhammad-ahmad 16208 Oct  9 10:39 ls-c
-rwxrwxr-x 1 muhammad-ahmad muhammad-ahmad 16208 Oct  9 10:40 ls-l
-rwxrwxr-x 1 muhammad-ahmad muhammad-ahmad 16216 Oct  2 15:48 redirection
```